

Create a Virtual Machine using Vagrant and Libvirt, set up an SSH server on it and connect via an SSH client; additionally, set up SSH key-based authentication use scp or rsync to transfer files securely between host and VM.

What is VM?

A virtual machine (VM) is a software-based, isolated emulation of a physical computer that runs its own operating system and applications. Simply imagine running Linux inside your Windows with separate RAM, SSD, etc.

It uses a **hypervisor** to divide physical hardware resources like CPU, memory, and storage from a host machine, allowing multiple, independent virtual computers to run on one physical server.

The hypervisor treats the physical hardware (host/your main OS) as a pool of resources. It carves up these resources and allocates them to various guest environments. Each guest OS behaves as if it has its own dedicated hardware, unaware that it is sharing the physical server with other VMs.

For example, suppose you have 1TB of storage, and you're using 2 VMs, suppose linux and mac inside your windows, so the job of hypervisor is to divide the 1TB storage so that each OS (your main windows, VM linux, VM mac) can get sufficient amount of storage, for an instance your main machine got 812GB, linux got 100GB and Mac got remaining 100GB. You can set the limit yourself. This goes the same for RAM, CPU, and other physical components.

In our task, we are strictly bound to use **vagrant** and **Libvirt**. Let's understand those libraries...

What is Libvirt?

Libvirt is an open-source API and management tool for managing platform virtualization. It is a stable, "low-level" manager that acts as a universal translator between you and the hypervisor.

What it does: It provides a consistent way to manage *different virtualization technologies* (KVM, QEMU, Xen, etc.). Instead of learning complex, hypervisor-specific commands, you use Libvirt to create, start, stop, and migrate VMs.

So different virtualization technologies ?? You might wonder what that means....

So the process of creating a VM is known as a virtualization technique, and yes, there are different ways to virtualize. The most famous ways are **KVM**, **QEMU**, and **Xen**. They work together.

You can Google them if you want to know how they work together, but I'm gonna skip this cause as I said, we're using **Libvirt** to manage different virtualization technologies, so you don't have to worry about them.

What is Vagrant?

Vagrant is a tool for building and maintaining portable, reproducible development environments. It is a "high-level" tool focused on workflow, not hardware management.

What it does: It allows you to define your entire VM setup (OS version, RAM, network, startup scripts) in a single text file called a **Vagrantfile**.

The Problem it Solves: "It works on my machine." With Vagrant, you can share a **Vagrantfile** with a clubmate, and when they run **vagrant up**, they get an identical environment to yours, regardless of whether they are on Linux, Mac, or Windows.

Now let's start....

First we need to install a couple of thing..

```
sudo apt install qemu-kvm libvirt-daemon-system libvirt-clients  
virt-manager bridge-utils
```

- **qemu-kvm:**
 - **KVM** is the engine (Hypervisor) that lets your Linux kernel run virtual machines at near-native speed.
 - **QEMU** is the "hardware architect" that emulates things like hard drives, USB ports, and video cards for the VM to use.
- **libvirt-daemon-system:** This is the background service (the "brain") that manages the VMs. Without this, there is no one to listen to Vagrant's requests.
- **libvirt-clients:** This provides command-line tools like **virsh**. This is how you (or Vagrant) talk to the daemon.
- **bridge-utils:** This allows the VM to connect to your host's network. It helps create the "virtual bridge" (the switch) so your VM can have its own IP

address.

- **virt-manager**: This is optional but highly recommended. It's a GUI (graphical interface) that lets you see the VM running. It's great for troubleshooting if the VM isn't booting correctly.

```
sudo apt update
sudo apt install vagrant
```

You know why we need this.

```
vagrant plugin install vagrant-libvirt
```

vagrant-libvirt Plugin:

Why? By default, Vagrant is built to talk to VirtualBox. Since we want to use the much faster, native KVM/Libvirt stack, we need this "translator" plugin.

By default, the Libvirt daemon is a "system-level" service. To manage VMs without typing **sudo** every five seconds, you need to add yourself to the authorized groups.

```
sudo adduser $USER libvirt
sudo adduser $USER kvm
```

Important: After running these commands, you **must log out and log back in** (or restart) for the permissions to apply to your current session.

Now that the foundation is laid, the hypervisor is active, Vagrant is installed, and the Libvirt "bridge" plugin is ready, we can move into the actual creation of your Virtual Machine.

Initialize the Project

Vagrant works on a "one folder = one environment" principle.

```
mkdir my_linux_vm && cd my_linux_vm
```

Generate the Vagrantfile:

```
vagrant init generic/ubuntu2204
```

*Note: `generic/ubuntu2204` is a popular, lightweight box that works exceptionally well with Libvirt.**

Configure the Vagrantfile:

By default, the *Vagrantfile* is filled with comments. We need to tell it two specific things: to use **Libvirt** and to give the VM a **static IP** so you can easily connect to it via your SSH client.

```
Vagrant.configure("2") do |config|
  config.vm.box = "generic/ubuntu2204"
  # Force the use of Libvirt
  config.vm.provider :libvirt do |lv|
    lv.memory = 1024
    lv.cpus = 1 end
  # Create a private network for SSH/SCP access
  # This acts like a virtual LAN cable between host and the VM
  config.vm.network "private_network", ip: "192.168.121.50"
end
```

This config worked for me.

Boot the Machine:

This command will download the OS image, create the VM in Libvirt, and automatically start the SSH server inside it.

```
vagrant up --provider=libvirt
```

After downloading, if you get something like...

```
default: Warning: Connection refused. Retrying...
default: Warning: Connection refused. Retrying...
default: Warning: Connection refused. Retrying...
default: Warning: Connection refused. Retrying...
default: Warning: Connection refused. Retrying...
```

```
default: Warning: Connection refused. Retrying...
```

A disconnected virtual switch likely causes it. Even though your "computer" (the VM) was turned on, it wasn't plugged into a network. In the world of Libvirt, virtual networks act like software-defined routers and switches.

Check:

```
sudo virsh net-list --all
```

It should show the `default` network as `active`. If now run:

```
# Start the network
sudo virsh net-start default

# Set it to start automatically when your system boots
sudo virsh net-autostart default
```

The Technical Breakdown

When you run a VM, it needs a way to talk to the host (your machine). Libvirt uses **Virtual Networks** (Bridge Interfaces) to make this happen.

1. **The `default` Network:** This is the standard virtual bridge (usually called `virbr0`) that Libvirt creates. It provides DHCP (assigning IP addresses) and DNS services to your VMs.
2. **The `inactive` State:** Because your `default` network was `inactive`, the virtual network cable from your VM was essentially plugged into a dead switch. The VM was booting up fine, but it couldn't get an IP address or "hear" Vagrant knocking on the door via SSH.
3. **Vagrant's "Retrying" Loop:** Vagrant was trying to connect to a specific IP, but since the network bridge was down, those packets had nowhere to go.

By running `sudo virsh net-start default`, you effectively "powered on" the virtual router.

Anyway, Let's understand some concepts while your image is downloading (I did that)...

SSH (Secure Shell) is a cryptographic network protocol that allows you to operate

network services securely over an unsecured network. In simpler terms, it creates a secure "tunnel" between your computer (the **Host**) and the Virtual Machine (the **Guest**) so you can run commands as if you were sitting right in front of it.

What is SSH used for?

Remote Command Execution: You can log into your VM from your terminal and run scripts, install software, or check logs.

Secure File Transfer: SSH provides the underlying security for tools like **SCP** (Secure Copy) and **SFTP**.

Tunneling/Port Forwarding: You can "forward" traffic from a web server running inside the VM to your own browser on your machine.

Security: It replaces older, insecure protocols like Telnet. All data (including your password) is encrypted, so anyone "sniffing" your network cannot see what you are doing.

After installing, run the following command to test the built-in connection:

```
vagrant ssh
```

If you see a prompt like `vagrant@ubuntu2204:~$`, you are inside your VM! Type `exit` to return to your terminal.

What are we gonna do in the upcoming steps?

When you set up SSH for your VM, we are moving through three distinct phases: **Establishing the Connection**, **Automating Identity**, and **Moving Data**.

When you run `vagrant ssh`, Vagrant uses a "private" insecure key that it generated during the boot process.

- **The Problem:** This only works from the folder where your `Vagrantfile` lives.
- **The Goal:** We want to connect using a standard SSH client (the `ssh` command) from *anywhere* on your system, using the IP address we set (`192.168.121.50`).

That's where the second part of our task comes in...

Manual SSH & Key-Based Auth

Generate a key on your host / actual machine:

```
ssh-keygen -t ed25519 -f ~/.ssh/vm_key
```

It will ask to create a password, remember it...

Push the public key to the VM:

Since Vagrant already has a temporary connection, we can "pipe" our key into the VM's authorized list:

```
cat ~/.ssh/vm_key.pub | vagrant ssh -c "mkdir -p ~/.ssh && cat >> ~/.ssh/authorized_keys"
```

Connect via SSH client:

```
ssh -i ~/.ssh/vm_key vagrant@192.168.121.50
```

Enter your password and hopin!

Last task of ours is SCP/Rsync to transfer any file from our host to the VM. Now that your key is set up, you can transfer files securely.

Transfer Files (SCP/Rsync):

Using SCP:

```
scp -i ~/.ssh/vm_key ~/Desktop/test.txt  
vagrant@192.168.121.50:/home/vagrant/
```

Using Rsync (Better for folders):

```
rsync -avz -e "ssh -i ~/.ssh/vm_key" ./local_folder/  
vagrant@192.168.121.50:~/remote_folder/
```

NOTE: If we go with basic `vagrant ssh`, these file transfers will be insecure...

Deploy your personal project on AWS Free Tier and make it publicly accessible.

Step 1: Build a project (website preferably)

Step 2: Host it

Ngl, you really don't get stuck in this, it's easy af..